



DNS MISCONFIGURATION

Bug Bounty Hunting Guide

Subdomain Takeover | Zone Transfer | Email Spoofing

DNSSEC | Dangling Records | Report Templates

2025 Edition

Table of Contents

- 1. Introduction to DNS Bugs
- 2. Subdomain Takeover
 - 2.1 CNAME Takeover
 - 2.2 NS Takeover
 - 2.3 MX Takeover
 - 2.4 A/AAAA Record Takeover
- 3. DNS Zone Transfer (AXFR)
- 4. Email Spoofing (SPF/DMARC/DKIM)
- 5. DNSSEC Misconfiguration
- 6. Dangling DNS Records
- 7. Wildcard DNS Abuse
- 8. TXT / SRV Record Leakage
- 9. DNS Rebinding
- 10. Internal IP Disclosure
- 11. CAA Record Issues
- 12. Tools & Automation
- 13. One-Liners
- 14. Report Templates
- 15. Tips & Tricks

1. Introduction to DNS Bugs

DNS misconfigurations are among the most common yet impactful vulnerabilities in bug bounty programs. They range from complete subdomain takeover to subtle email spoofing misconfigurations. Unlike application-level bugs, DNS issues often affect entire domains or subdomains, making them high-impact findings.

Vulnerability	Severity	Bounty Range
Subdomain Takeover Critical	P1/P2	\$500 - \$10,000+
DNS Zone Transfer High	P2	\$250 - \$2,500
Email Spoofing (No DMARC) Medium	P3/P4	\$100 - \$1,000
DNSSEC Misconfiguration Medium	P3/P4	\$100 - \$500
Dangling CNAME Records High	P2/P3	\$250 - \$1,500
Internal IP Disclosure Low	P5	\$50 - \$250
TXT Record Leakage Low	P5	\$50 - \$200
Wildcard DNS Abuse Medium	P3/P4	\$100 - \$750
CAA Bypass Medium	P3/P4	\$100 - \$500

2. Subdomain Takeover

2.1 CNAME Takeover

The most common and highest-paying DNS bug. A subdomain has a CNAME pointing to a third-party service (GitHub Pages, Heroku, AWS S3, Shopify, etc.) that no longer exists. An attacker can claim the service name and serve content under the victim's subdomain.

Step-by-Step Reproduction

```
# Step 1: Enumerate subdomains
subfinder -d target.com -silent | tee subs.txt

# Step 2: Check CNAME records
cat subs.txt | while read sub; do
    cname=$(dig +short CNAME "$sub" | head -1)
    if [ -n "$cname" ]; then
        echo "$sub -> $cname"
    fi
done | tee cnames.txt

# Step 3: Identify vulnerable CNAMEs
# Common takeover signatures:
# *.github.io -> Create GitHub repo with same name
# *.herokuapp.com -> Create Heroku app with same name
# *.azurewebsites.net -> Create Azure app
# *.cloudfront.net -> Check if distribution deleted
# *.s3.amazonaws.com -> Check if bucket exists
# *.shopify.com -> Check if store available
# *.fastly.net -> Check if service deleted
# *.netlify.app -> Create Netlify site
# *.vercel.app -> Create Vercel project
# *.firebaseapp.com -> Create Firebase project
# *.elasticbeanstalk.com -> Create EB environment
# *.surge.sh -> Create Surge site
# *.glitch.me -> Check availability
# *.pagely.net -> Check availability
# *.myshopify.com -> Check store availability
# *.wordpress.com -> Check blog availability
# *.tumblr.com -> Check blog availability
# *.strikingly.com -> Check site availability
# *.ghbtns.com -> Check availability
# *.helpjuice.com -> Check knowledge base availability
# *.helpscoutdocs.com -> Check docs availability
# *.launchrock.com -> Check availability
# *.pingdom.com -> Check availability
# *.tilda.ws -> Check availability
# *.unbouncepages.com -> Check availability
# *.wpeengine.com -> Check availability

# Step 4: Verify takeover with HTTP request
curl -s -o /dev/null -w "%{http_code}" http://vulnerable.target.com/
# Look for: 404, NXDOMAIN in response, or service-specific error

# Step 5: Claim the resource (if available)
# Example: GitHub Pages
# Create GitHub repository named "vulnerable"
# Add CNAME file with "vulnerable.target.com"
# Enable GitHub Pages
# Wait for DNS propagation
```

Report Template: Subdomain Takeover

```
Subdomain Takeover - vulnerable.target.com
High (P2)
https://vulnerable.target.com
```

The subdomain vulnerable.target.com has a CNAME record pointing to attacker.github.io, but the GitHub repository no longer exists. An attacker can claim this repository and serve arbitrary content under target.com's subdomain.

1. Query DNS for CNAME of vulnerable.target.com:

```
dig CNAME vulnerable.target.com
```

```
# Response: vulnerable.target.com. CNAME attacker.github.io.
```
 2. Check if the GitHub Pages site exists:

```
curl -I http://attacker.github.io/
```

```
# Returns 404 - Not Found
```
 3. Claim the repository by creating "attacker.github.io" on GitHub and adding CNAME file: vulnerable.target.com
 4. Verify content is served:

```
curl http://vulnerable.target.com/
```

```
# Shows attacker-controlled content
```

 - Cookie theft via XSS
 - Session hijacking (if cookies lack proper flags)
 - Phishing attacks under target.com domain
 - SSL certificate trust abuse
 - Reputation damage
1. Remove the CNAME record: vulnerable.target.com
 2. Or reclaim the resource on the third-party platform
 3. Audit all DNS records for similar dangling CNAMEs
 4. Implement DNS monitoring for unused records

2.2 NS Takeover

If a subdomain delegates its DNS to external nameservers (NS records) and those nameservers are no longer controlled by the owner, an attacker can take control of the entire subdomain's DNS zone.

```
# Find NS records for target
dig +short NS target.com
dig +short NS subdomain.target.com

# Check if NS servers respond authoritatively
dig @ns1.external-dns.com subdomain.target.com AXFR

# Check if nameserver domain is expired
whois ns1.external-dns.com | grep "Expiry Date\|Expiration"

# Check if nameserver IP belongs to target
nslookup ns1.external-dns.com
```

Report Template: NS Takeover

DNS Zone Delegation Takeover
Critical (P1)

The subdomain dev.target.com delegates DNS to ns1.expired-service.com which is no longer owned by the organization. An attacker can register this domain and control all DNS records for dev.target.com.

1. dig +short NS dev.target.com
Returns: ns1.expired-service.com
2. whois expired-service.com
Domain is expired and available for registration
3. Register expired-service.com
4. Configure ns1.expired-service.com as authoritative
5. Add any DNS record for dev.target.com

- Full DNS control over dev.target.com
- Redirect traffic to attacker servers
- SSL certificate issuance via ACME DNS-01
- Email interception via MX record manipulation

2.3 MX Takeover

```
# Check MX records
dig +short MX target.com
dig +short MX mail.target.com

# If MX points to a third-party email service that no longer exists:
# Example: MX 10 mail.sendgrid.net (expired account)
# Attacker can potentially claim the email service account

# Check email service existence
nc -zv mail.sendgrid.net 25
curl -v telnet://mail.sendgrid.net:25
```

2.4 A/AAAA Record Takeover

When an A record points to an IP that no longer belongs to the target (e.g., cloud instance terminated, VPS expired), an attacker who controls that IP can serve content.

```
# Check A records for subdomains
cat subs.txt | while read sub; do
  ip=$(dig +short A "$sub" | head -1)
  if [ -n "$ip" ]; then
    # Check if IP responds with target content
    resp=$(curl -s -H "Host: $sub" --max-time 5 "http://$ip/" 2>/dev/null | head -1)
    echo "$sub -> $ip -> $resp"
  fi
done

# Check if IP is owned by target (ASN lookup)
curl -s "https://ipinfo.io/$ip/org"

# Check if IP responds to different Host headers
curl -s -H "Host: evil.com" http://$ip/ | head -10
```

3. DNS Zone Transfer (AXFR)

A zone transfer allows a DNS server to replicate its entire zone file to another server. If misconfigured, anyone can request the full zone file, revealing all subdomains, internal IPs, mail servers, and more.

3. DNS Zone Transfer (AXFR)

```
# Step 1: Find nameservers
dig +short NS target.com

# Step 2: Attempt zone transfer from each nameserver
dig @ns1.target.com target.com AXFR
dig @ns2.target.com target.com AXFR

# Step 3: Use host command for zone transfer
host -l target.com ns1.target.com
host -la target.com ns1.target.com

# Step 4: Automated with dnsrecon
dnsrecon -d target.com -t axfr

# Step 5: Use fierce
fierce --domain target.com --dns-servers ns1.target.com

# Step 6: Use dig for specific record types in zone
for ns in $(dig +short NS target.com); do
    echo "=== Testing $ns ==="
    dig @$ns target.com AXFR
done

# Step 7: Zone transfer via IPv6
dig @2001:db8::1 target.com AXFR

# Step 8: Check for AXFR on subdomains
dig @ns1.target.com subdomain.target.com AXFR
```


Report Template: DNS Zone Transfer

DNS Zone Transfer Misconfiguration (AXFR)
High (P2)

The nameserver ns1.target.com allows unrestricted DNS zone transfers, permitting any attacker to retrieve the complete DNS zone file for target.com, including all subdomains, internal IP addresses, and service records.

```
1. dig +short NS target.com
   # ns1.target.com, ns2.target.com

2. Attempt zone transfer:
   dig @ns1.target.com target.com AXFR

3. Result: Full zone file is returned:
;target.com.      IN  AXFR
target.com.      300 IN  A   1.2.3.4
www.target.com.  300 IN  CNAME target.com.
mail.target.com. 300 IN  A   10.0.0.5
vpn.target.com.  300 IN  A   192.168.1.10
...
```

- Complete infrastructure enumeration
- Internal IP address disclosure
- Attack surface expansion
- Service identification for targeted attacks
- Private subdomain exposure

1. Restrict AXFR to authorized secondary nameservers only
2. Configure BIND: allow-transfer { trusted-ns; };
3. Use TSIG authentication for zone transfers
4. Monitor for unauthorized AXFR attempts

4. Email Spoofing (SPF/DMARC/DKIM)

4.1 SPF Misconfiguration

```
# Check SPF record
dig +short TXT target.com | grep "v=spf1"

# Common SPF misconfigurations:
# v=spf1 +all      -> Allows ANY server to send email (CRITICAL)
# v=spf1 ?all      -> Neutral (no enforcement)
# v=spf1 ~all      -> Soft fail (only marks, doesn't reject)
# v=spf1 include:_spf.google.com +all -> +all overrides everything
# v=spf1 mx +all    -> +all allows all
# v=spf1 a ip4:0.0.0.0/0 -> Allows any IP
# Missing SPF record entirely

# Parse SPF record
python3 -c "
import dns.resolver
answers = dns.resolver.resolve('target.com', 'TXT')
for rdata in answers:
    txt = str(rdata).strip('\n')
    if 'v=spf1' in txt:
        print('SPF:', txt)
        if '+all' in txt: print('CRITICAL: +all allows any sender')
        elif '~all' in txt: print('WARNING: soft fail only')
        elif '-all' not in txt: print('WARNING: no strict -all')
"
```

```
# Test SPF with online tool or custom script
curl -s "https://api.kickbox.com/v2/verify?email=test@target.com&apikey=KEY"

# Send test spoofed email (use mail-tester.com or similar)
# Or use swaks:
swaks --to victim@example.com --from admin@target.com \
  --header "Subject: Test" --body "Testing SPF" \
  --server target.com.smtp-provider.com
```

4.2 DMARC Misconfiguration

```
# Check DMARC record
dig +short TXT _dmarc.target.com

# Check for missing DMARC
dig +short TXT _dmarc.target.com | grep "v=DMARC1"

# Weak DMARC configurations:
# v=DMARC1; p=none;           -> No enforcement (only reports)
# v=DMARC1; p=quarantine;    -> Moderate (spam folder)
# v=DMARC1; p=reject;        -> Strong (recommended)
# Missing record = no protection

# Check DMARC policy with dmarcian
curl -s "https://dmarcian.com/dmarc-inspector/?domain=target.com"

# Check with Google Admin Toolbox
# https://toolbox.googleapps.com/apps/checkmx/
```

4.3 DKIM Misconfiguration

```
# Check DKIM selectors
dig +short TXT default._domainkey.target.com
dig +short TXT dkim._domainkey.target.com
dig +short TXT selector1._domainkey.target.com

# Common issues:
# - Missing DKIM entirely
# - DKIM key too short (512-bit or 768-bit)
# - Revoked/expired DKIM keys still in DNS
# - Selector enumeration revealing keys

# Enumerate DKIM selectors
cat dkim_selectors.txt | while read selector; do
    result=$(dig +short TXT "${selector}._domainkey.target.com")
    if [ -n "$result" ]; then
        echo "Found: $selector -> $result"
    fi
done

# Common selectors to test:
# default, dkim, selector1, selector2, google, mail, s1, s2, m1, m2
# mx1, smtp, k1, k2, key1, key2, mailjet, sendgrid, mandrill
# amazonSES, zoho, protonmail, o365, office365
```

Report Template: Email Spoofing

Issue: Missing/Weak DMARC Enables Email Spoofing
Severity: Medium (P3)

The domain target.com lacks a DMARC record with enforcement policy, allowing attackers to send spoofed emails that pass SPF alignment checks and appear to originate from target.com.

1. Check DMARC record:

```
dig TXT _dmarc.target.com
# No DMARC record found
```

2. Check SPF record:

```
dig TXT target.com | grep spf
# v=spf1 include:_spf.google.com ~all
```

3. Verify email can be spoofed:

- Use mail-tester.com or similar service
- Send email from attacker@target.com
- Email passes SPF (due to ~all soft fail)
- No DMARC to reject the message

4. Check deliverability:

```
# Email lands in inbox with target.com From address
```

- Phishing attacks using target.com domain
- Business Email Compromise (BEC)
- Credential theft via spoofed emails
- Reputation damage
- Customer trust erosion

1. Add DMARC record: _dmarc.target.com

```
TXT "v=DMARC1; p=reject; rua=mailto:dmarc@target.com; ruf=mailto:dmarc@target.com; fo=1"
```

2. Ensure SPF uses -all (hard fail):

```
"v=spf1 include:_spf.google.com -all"
```

3. Implement DKIM signing for all outgoing mail

4. Monitor DMARC reports before enforcing reject policy

5. DNSSEC Misconfiguration

```
# Check if DNSSEC is enabled
dig +dnssec target.com SOA | grep -E "RRSIG|NSEC|NSEC3|ad|cd"

# Check DNSKEY records
dig +short DNSKEY target.com

# Verify DNSSEC chain
delv target.com SOA
# or
drill -S target.com

# Check for DNSSEC validation failures
dig @8.8.8.8 +dnssec target.com

# Common misconfigurations:
# - DNSSEC enabled but signatures expired
# - DNSSEC enabled but not propagated to parent zone
# - Missing DS record at registrar
# - Algorithm downgrade vulnerabilities
# - Key rollover failures
# - NSEC walking (zone enumeration via NSEC records)

# NSEC walking (enumerate zone if NSEC is used)
dig +dnssec target.com NSEC

# Use ldns-walk for NSEC enumeration
ldns-walk target.com

# Use nsec3walker for NSEC3 hashed enumeration
python3 nsec3walker.py target.com

# Check for algorithm vulnerabilities
dig +short DNSKEY target.com | awk '{print $3}' | sort -u
# 5 = RSASHA1 (deprecated)
# 8 = RSASHA256 (recommended)
# 13 = ECDSAP256SHA256 (recommended)
# 14 = ECDSAP384SHA384
```

Report Template: DNSSEC Misconfiguration

DNSSEC Validation Failure / Missing DS Record
Medium (P3)

DNSSEC is configured on target.com but the DS record is missing from the parent zone (.com), causing validation failures and allowing potential DNS cache poisoning attacks.

1. Check DNSKEY records exist:
dig DNSKEY target.com
DNSKEY records present
 2. Check DS record at parent:
dig DS target.com @a.gtld-servers.net
No DS record found
 3. Verify validation failure:
delv target.com A
Resolution failed: no valid signature found
- DNS cache poisoning vulnerability
 - Man-in-the-middle via DNS spoofing
 - Users cannot validate DNS responses
 - Downgrade to insecure DNS
1. Upload DS record to domain registrar
 2. Ensure DS digest matches DNSKEY
 3. Use algorithm 8 (RSASHA256) or 13 (ECDSAP256SHA256)
 4. Monitor DNSSEC validation with dnsviz.net

6. Dangling DNS Records

A dangling DNS record points to a resource that no longer exists. Unlike full subdomain takeover, dangling records may only cause information disclosure or partial service disruption.

```

# Find all DNS records for target
dig ANY target.com +short
dig ANY *.target.com +short

# Check for dangling records with dnsrecon
dnsrecon -d target.com -t std

# Use dnsenum for comprehensive enumeration
dnsenum target.com

# Common dangling record types:
# CNAME pointing to deleted service
# A record pointing to terminated cloud instance
# MX record pointing to closed email service
# PTR record pointing to released IP
# SRV record for decommissioned service

# Verify each IP still belongs to target
for ip in $(dig +short A target.com; dig +short A www.target.com); do
    org=$(curl -s "https://ipinfo.io/$ip/org" 2>/dev/null)
    echo "$ip -> $org"
done

# Check if MX server still accepts mail
for mx in $(dig +short MX target.com | awk '{print $2}'); do
    echo "Testing $mx..."
    (echo "QUIT" | nc -w 5 "$mx" 25)
done

# Bulk dangling check with amass + httpx
amass enum -d target.com -o amass_subs.txt
# Check which resolve but don't respond
cat amass_subs.txt | dnsx -silent | httpx -silent -mc 0

```

Report Template: Dangling DNS Record

Dangling CNAME Record - old.target.com
Medium (P3)

The subdomain old.target.com has a CNAME pointing to old-app.herokuapp.com which no longer exists. While not fully takeable, this causes information disclosure about previously used infrastructure and potential subdomain hijacking risk.

1. dig CNAME old.target.com
old.target.com -> old-app.herokuapp.com

2. Check Heroku app existence:
curl -I https://old-app.herokuapp.com/
HTTP 404 - No such app

3. Verify subdomain responds with Heroku error:
curl -I https://old.target.com/
Shows Heroku "No such app" page

- Information disclosure (previously used Heroku)
- Subdomain takeover risk if service is re-registered
- User confusion / degraded experience
- Potential for chained attacks

1. Remove the CNAME record: old.target.com
2. Or reclaim the Heroku app name
3. Audit DNS monthly for dangling records
4. Use dangling DNS detection tools

7. Wildcard DNS Abuse

Wildcard DNS records (*.target.com) respond with the same IP for any non-existent subdomain. This can be abused for subdomain enumeration bypass, cookie scope expansion, and certificate issuance manipulation.


```

# Check for wildcard DNS
dig +short A random123456.target.com
dig +short A testwildcard.target.com

# If both resolve to the same IP, wildcard is configured

# Abuse scenarios:

# 1. Subdomain enumeration bypass
# Tools may report all subdomains as "alive" due to wildcard
# Use wildcard detection:
subfinder -d target.com -silent | dnsx -silent | \
  while read sub; do
    ip=$(dig +short A "$sub" | head -1)
    wild_ip=$(dig +short A "random123.$sub" | head -1)
    if [ "$ip" = "$wild_ip" ]; then
      echo "WILDCARD: $sub -> $ip"
    else
      echo "REAL: $sub -> $ip"
    fi
  done

# 2. Cookie scope abuse
curl -s https://random.target.com/ -c cookies.txt
curl -s https://real.target.com/ -b cookies.txt
# Cookies set on *.target.com scope are sent to all subdomains

# 3. Certificate issuance abuse (if ACME DNS-01 is used)
# Request certificate for *.target.com
# Any subdomain will resolve, enabling validation

# 4. Phishing with any subdomain
curl -s https://support-security.target.com/
# Resolves to same IP as www.target.com
# User sees legitimate site at deceptive subdomain

# Wildcard + CDN interaction
curl -s -I https://anything.target.com/ | grep "CF-Ray\|X-Cache"
# All subdomains cached by CDN

```

Report Template: Wildcard DNS Abuse

Wildcard DNS Enables Phishing and Enumeration Issues
Medium (P3)

The domain target.com has a wildcard DNS record (*.target.com) that resolves any non-existent subdomain to the main application. This enables phishing attacks using deceptive subdomains and complicates security monitoring.

1. Query random non-existent subdomain:

```
dig A security-update.target.com
```

 # Resolves to 1.2.3.4 (same as www.target.com)
2. Access deceptive subdomain:

```
curl -I https://security-update.target.com/
```

 # HTTP 200 - serves main application
3. Verify wildcard record:

```
dig A *.target.com
```

 # *.target.com -> 1.2.3.4

- Phishing with deceptive subdomains
- Subdomain enumeration confusion
- Cookie scope abuse
- Social engineering under trusted domain
- Monitoring blind spots

1. Remove wildcard A record (*.target.com)
2. Explicitly define only required subdomains
3. Monitor certificate transparency for wildcard certs
4. Implement subdomain monitoring

8. TXT / SRV Record Leakage

```
# Enumerate all TXT records
dig +short TXT target.com
dig +short TXT _acme-challenge.target.com
dig +short TXT _dmarc.target.com
dig +short TXT _domainkey.target.com
dig +short TXT _verification.target.com
dig +short TXT _github-challenge.target.com
dig +short TXT _amazonses.target.com

# Search for sensitive information in TXT records
dig +short TXT target.com | grep -iE "key|token|secret|password|aws|azure|api"

# Check SRV records for service discovery
dig +short SRV _sip._tcp.target.com
dig +short SRV _xmpp-server._tcp.target.com
dig +short SRV _ldap._tcp.target.com
dig +short SRV _kerberos._tcp.target.com

# Check for service discovery via DNS
dig +short SRV _minecraft._tcp.target.com
dig +short SRV _ts3._udp.target.com

# Amazon SES verification (shows AWS usage)
dig +short TXT _amazonses.target.com

# Google verification
dig +short TXT google._domainkey.target.com

# Microsoft verification
dig +short TXT MS=ms12345678.target.com

# Atlassian verification
dig +short TXT atlassian-domain-verification=target.com

# SendGrid
dig +short TXT s1._domainkey.target.com

# Check for DKIM keys (may be weak or exposed)
dig +short TXT default._domainkey.target.com | base64 -d 2>/dev/null | openssl rsa -
pubin -text -noout 2>/dev/null

# Full TXT enumeration with dnsrecon
dnsrecon -d target.com -t zonewalk
dnsrecon -d target.com -t tld
```

Report Template: TXT Record Information Disclosure

TXT Record Exposes Internal Information
Low (P5)

DNS TXT records for target.com contain internal verification codes and infrastructure identifiers that aid attacker reconnaissance and provide unnecessary information disclosure.

```
1. dig TXT target.com
# "MS=ms12345678" (Microsoft tenant ID)
# "google-site-verification=abc123" (Google property)
# "atlassian-domain-verification=xyz789" (Atlassian)
```

- Infrastructure enumeration
- Third-party service identification
- Potential tenant/targeted attack surface
- Information aiding social engineering

1. Remove verification records after use
2. Audit and minimize TXT records
3. Move sensitive config to internal DNS
4. Regular DNS record review

9. DNS Rebinding

DNS rebinding allows an attacker to bypass Same-Origin Policy by making a domain first resolve to an attacker IP, then to an internal IP after the browser's DNS cache expires.

```

# Concept: Attacker controls DNS for evil.com
# Phase 1: evil.com -> attacker IP (serves malicious JS)
# Phase 2: After cache expires, evil.com -> 192.168.1.1 (internal IP)
# Browser makes request to internal IP under same origin

# Check target's DNS TTL (time to live)
dig target.com | grep "Query time\|ANSWER SECTION" -A 5

# Low TTL (< 60 seconds) makes rebinding more practical
dig +short target.com | awk '{print $2}' # TTL value

# Tools for DNS rebinding:
# - Singularity (https://github.com/nccgroup/singularity)
# - Whonow (https://github.com/brannondorsey/whonow)
# - rbndr.us (public DNS rebinding service)
# - dnsrebind (https://github.com/mpgn/dnsrebind)

# Using rbndr.us for quick testing:
# Format: -.rbndr.us
# Example: 1.2.3.4-192.168.1.1.rbndr.us

# Test if application makes requests to user-provided domains
curl -X POST https://target.com/api/fetch \
  -H "Content-Type: application/json" \
  -d '{"url":"http://1.2.3.4-192.168.1.1.rbndr.us/"}'

# Check WebSocket connection to attacker-controlled domain
# After rebind, WebSocket connects to internal IP

# Check if SSRF protection trusts internal domains
curl -X POST https://target.com/api/webhook \
  -d '{"url":"http://mydomain.com"}'
# Where mydomain.com resolves to attacker then to internal IP

```

Report Template: DNS Rebinding Vulnerability

Title: SSRF via DNS Rebinding
Severity: High (P2)

The application at target.com accepts user-provided URLs for webhook/web scraping functionality. Combined with the low DNS TTL of the target domain, an attacker can use DNS rebinding to bypass SSRF protections and access internal services.

1. Check target.com TTL:
`dig target.com | grep -A1 "ANSWER SECTION"`
 # TTL: 30 seconds
 2. Set up rebinding DNS (using singularity or rbndr.us)
 3. Register webhook:
`POST /api/webhook`
`{"url":"http://attacker-192.168.1.1.rbndr.us/internal"}`
 4. First request resolves to attacker IP (serves JS)
 5. After 30s, resolves to 192.168.1.1
 6. Browser/application fetches internal service
- SSRF bypass
 - Internal network access
 - Cloud metadata extraction
 - Port scanning from target server
 - Attack on internal services
1. Validate and block internal IP ranges (RFC 1918)
 2. Resolve DNS before request and validate IP
 3. Disable follow redirects for user URLs
 4. Use URL whitelist for allowed domains

10. Internal IP Disclosure

```
# Check for internal IPs in DNS responses
dig +short target.com
dig +short www.target.com

# Internal IP ranges:
# 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 (RFC 1918)
# 127.0.0.0/8 (loopback)
# 169.254.0.0/16 (link-local)
# 100.64.0.0/10 (CGNAT)

# PTR records often reveal internal naming
dig +short -x 10.0.0.5
# May return: internal-web-server.corp.target.com

# Check MX records for internal IPs
dig +short MX target.com
# If MX points to 192.168.x.x, internal mail server exposed

# Check NS records for internal IPs
dig +short NS target.com
# If NS is 10.x.x.x, internal DNS exposed

# Zone transfer often reveals internal IPs
dig @ns1.target.com target.com AXFR | grep -E "10\.|172\.(1[6-9]|2[0-9]|3[01])\.|192
\.168\."

# Automated internal IP detection in DNS
dig +short ANY target.com | grep -E "^(10\.|172\.(1[6-9]|2[0-9]|3[01])\.|192\.168\.|
127\.|169\.254\.|100\.(6[4-9]|[7-9][0-9]|1[0-1][0-9]|12[0-7])\.)"
```

Report Template: Internal IP Disclosure via DNS

Internal IP Address Exposed in DNS Records
Low (P5)

DNS records for target.com contain RFC 1918 private IP addresses, exposing internal network topology and aiding attacker reconnaissance.

1. Query A records:
dig A internal.target.com
internal.target.com -> 192.168.1.10
2. Query MX records:
dig MX target.com
mail.target.com -> 10.0.0.5
3. Query zone file (if AXFR available):
dig @ns1.target.com target.com AXFR
Multiple internal IPs revealed

- Internal network topology exposure
- Attack surface identification
- Aid for VPN/gateway targeting
- Social engineering intelligence

1. Replace internal IPs with NAT/public IPs in external DNS
2. Use split-horizon DNS (internal vs external views)
3. Remove internal-only records from public DNS
4. Audit DNS for RFC 1918 addresses

11. CAA Record Issues

Certification Authority Authorization (CAA) records specify which certificate authorities are allowed to issue certificates for a domain. Missing or weak CAA records can allow unauthorized certificate issuance.


```
# Check CAA record
dig +short CAA target.com

# Good CAA configuration:
# 0 issue "letsencrypt.org"
# 0 issue "digicert.com"
# 0 issuewild ";" (no wildcard certs)
# 0 iodef "mailto:security@target.com"

# Weak/missing CAA:
# No CAA record at all -> Any CA can issue
# 0 issue ";" -> No certificates allowed (breaks TLS)
# 0 issuewild "letsencrypt.org" -> Wildcards allowed by LE

# Check for CAA at parent and subdomain levels
dig +short CAA target.com
dig +short CAA www.target.com
dig +short CAA *.target.com

# CAA reporting (iodef)
dig +short CAA target.com | grep iodef

# CAA checking with certspotter
curl -s "https://api.certspotter.com/v1/issuances?domain=target.com&match_wildcards=true&expand=dns_names" | \
jq '.[].issuer.name' | sort -u
```

Report Template: Missing CAA Record

Missing CAA Record Allows Unauthorized Certificate Issuance Medium (P3)

The domain target.com lacks a CAA DNS record, allowing any certificate authority to issue certificates for the domain without authorization.

1. Check CAA record:

```
dig CAA target.com
# No CAA record found
```

2. Check certificate issuance history:

```
curl "https://crt.sh/?q=target.com&output=json"
# Multiple CAs have issued certificates
```

- Unauthorized certificate issuance
- Man-in-the-middle potential
- Rogue CA compromise exposure
- Certificate transparency bypass

1. Add CAA record:

```
target.com CAA 0 issue "letsencrypt.org"
target.com CAA 0 issue "digicert.com"
target.com CAA 0 issuewild ";"
target.com CAA 0 iodef "mailto:security@target.com"
```

2. Monitor certificate transparency logs

3. Subscribe to CAA violation alerts

12. Tools & Automation

Tool	Purpose	Command
dig	DNS query	<code>dig +short A target.com</code>
host	DNS lookup / zone transfer	<code>host -l target.com ns1.target.com</code>
nslookup	Interactive DNS	<code>nslookup -type=ANY target.com</code>
dnsrecon	DNS enumeration	<code>dnsrecon -d target.com -t axfr</code>
dnsenum	DNS enumeration + brute	<code>dnsenum target.com</code>
fierce	DNS recon / zone walk	<code>fierce --domain target.com</code>
amass	Deep subdomain enum	<code>amass enum -d target.com</code>
subfinder	Fast subdomain discovery	<code>subfinder -d target.com</code>
dnsx	Fast DNS resolver	<code>cat subs dnsx -a -resp</code>
whois	Domain registration data	<code>whois target.com</code>
delv	DNSSEC validation	<code>delv target.com SOA</code>
ldns-walk	NSEC zone walking	<code>ldns-walk target.com</code>
nmap	DNS scripts	<code>nmap --script dns-* target.com</code>
massdns	High-speed DNS resolver	<code>massdns -r resolvers.txt subs.txt</code>
cloud_enum	Cloud service enum	<code>python3 cloud_enum.py -k target.com</code>
dnstwist	Typosquat detection	<code>dnstwist target.com</code>
dnsvalidator	Resolver validation	<code>dnsvalidator -tL resolvers.txt</code>

13. One-Liners

Complete DNS Recon Pipeline

```
# Full DNS enumeration pipeline
subfinder -d target.com -silent | \
  dnsx -a -aaaa -cname -mx -txt -ns -soa -ptr -srv -resp -o dns_records.jsonl

# Extract CNAMEs for takeover analysis
cat dns_records.jsonl | jq -r 'select(.cname!=null) | "\(.host) -> \(.cname)"' | \
  while read line; do
    cname=$(echo "$line" | awk -F' -> ' '{print $2}')
    status=$(curl -s -o /dev/null -w "%{http_code}" --max-time 5 "http://$cname/" 2>/dev/null)
    if [ "$status" = "404" ] || [ "$status" = "000" ]; then
      echo "POTENTIAL TAKEOVER: $line (HTTP $status)"
    fi
  done
```

Passive subfinder dnsx

Zone Transfer + Enumeration

```
# Try AXFR on all discovered nameservers
for ns in $(dig +short NS target.com); do
  echo "=== Testing $ns ==="
  dig @$ns target.com AXFR | head -50
done

# Using dnsrecon for comprehensive check
dnsrecon -d target.com -t axfr,zonewalk,std -x /tmp/dnsrecon.xml
```

Active dnsrecon

Email Security Audit

```
# Complete SPF/DMARC/DKIM audit
echo "=== SPF ==="
dig +short TXT target.com | grep "v=spf1"

echo "=== DMARC ==="
dig +short TXT _dmarc.target.com

echo "=== DKIM ==="
for selector in default dkim selector1 selector2 google mail; do
    result=$(dig +short TXT "${selector}._domainkey.target.com")
    [ -n "$result" ] && echo "$selector: $result"
done

# Automated email security scoring
curl -s "https://api.mailcheck.ai/domain/target.com" | jq
```

Passive dig

Subdomain Takeover Mass Scanner

```
# Mass subdomain takeover detection
#!/bin/bash
DOMAIN="target.com"
subfinder -d "$DOMAIN" -silent | tee subs.txt

cat subs.txt | dnsx -cname -resp -silent | tee cnames.txt

cat cnames.txt | while read line; do
    sub=$(echo "$line" | awk '{print $1}')
    cname=$(echo "$line" | awk '{print $2}')

    # Check if CNAME resolves
    cname_ip=$(dig +short A "$cname" | head -1)

    if [ -z "$cname_ip" ]; then
        echo "[TAKEOVER] $sub -> $cname (NXDOMAIN)"
    else
        # Check HTTP response
        status=$(curl -s -o /dev/null -w "%{http_code}" --max-time 5 "http://$cname/" 2>/dev/null)
        if [ "$status" = "404" ]; then
            echo "[TAKEOVER] $sub -> $cname (HTTP 404)"
        fi
    fi
done | tee takeover_candidates.txt
```

Active bash

Internal IP Detection in DNS

```
# Find all RFC 1918 IPs in DNS records
dig +short ANY target.com | \
    grep -E "(10\.|172\.(1[6-9]|2[0-9]|3[01])\.|192\.168\.|127\.|169\.254\.)"

# Mass check for internal IPs across all record types
dnsx -d target.com -a -aaaa -cname -mx -ns -resp | \
    jq -r 'select(.a[]? | test("^(10\\.|172\\. (1[6-9]|2[0-9]|3[01])\\.|192\\.168\\.|127\\.|169\\.254\\.))" | \(.host): \(.a | join(", "))')

# Check PTR records for internal naming
dig +short -x 10.0.0.5
dig +short -x 192.168.1.1
```

Passive dnsx

Dangling Record Detector

```
# Find dangling A records (IP doesn't respond or serves different content)
cat subs.txt | while read sub; do
    ip=$(dig +short A "$sub" | head -1)
    [ -z "$ip" ] && continue

    # Check if IP belongs to known cloud provider
    org=$(curl -s "https://ipinfo.io/$ip/org" 2>/dev/null)
    echo "$sub -> $ip ($org)"

    # Test HTTP with proper Host header
    resp=$(curl -s -H "Host: $sub" --max-time 5 "http://$ip/" 2>/dev/null | head -
1)
    echo "  Response: $resp"
done
```

Passive bash

14. Report Templates

Generic DNS Bug Report Structure

Title: [Vulnerability Type] - [Affected Domain/Subdomain]
Severity: [P1/P2/P3/P4/P5]
Assets: [target.com / specific subdomain]

Summary

One or two sentences describing the bug and its impact.

Steps to Reproduce

1. [Command or action]
2. [Expected result showing vulnerability]
3. [Verification step]

Impact

- Bullet points of specific consequences
- Business impact
- User impact

Proof of Concept

[Screenshots, commands, or output demonstrating the bug]

Recommendations

1. [Specific fix]
2. [Prevention measure]
3. [Monitoring recommendation]

References

- [Relevant RFC, CWE, or security guide]
- [Tool documentation]

Severity Classification Guide

Vulnerability	Severity	Rationale
Subdomain Takeover (MX/NS)	P1 - Critical	Full email/DNS control, phishing, certificate issuance
Subdomain Takeover (CNAME)	P2 - High	Content control, phishing, cookie theft
DNS Zone Transfer	P2 - High	Complete infrastructure disclosure
Missing DMARC + SPF +all	P2 - High	Direct email spoofing capability
Missing DMARC (only)	P3 - Medium	Spoofing possible but may be caught
SPF soft fail (~all)	P3 - Medium	Partial protection, spoofing still possible
Dangling CNAME	P3 - Medium	Information disclosure + takeover risk
DNSSEC Missing DS	P3 - Medium	Cache poisoning risk
Wildcard DNS	P3 - Medium	Phishing, enumeration issues
TXT Record Leakage	P4/P5 - Low	Information disclosure only
Internal IP Disclosure	P5 - Informational	Reconnaissance aid only
Weak DKIM Key	P4 - Low	Theoretical key forgery
Missing CAA	P3 - Medium	Unauthorized certificate issuance

15. Tips & Tricks

Tip 1: Always Verify Takeover Before Reporting

A CNAME pointing to a deleted GitHub Pages site is only a valid takeover if you can actually claim the repository name. Some "deleted" services are actually just private or renamed. Always attempt to claim the resource to prove takeover capability.

Tip 2: Check Parent Domain for Wildcards

Before reporting a subdomain takeover, check if *.target.com is a wildcard. If it is, the "subdomain" may not actually have its own DNS record - it's just the wildcard responding. A true takeover requires a dedicated CNAME or A record.

Tip 3: Test MX Takeover with Email

For MX takeover claims, actually send an email to the subdomain and verify delivery. If the MX server is down but the domain is not claimable, it's a dangling record (P3) not a full takeover (P1/P2).

Tip 4: Use DNSSEC Validation Tools

Use dnsviz.net and zonemaster.net for comprehensive DNSSEC analysis. These tools provide visual reports that are excellent for bug bounty reports.

Tip 5: Monitor Certificate Transparency for DNS Issues

Sudden certificate issuance for subdomains you discovered can indicate someone else found the same issue. Report quickly. Tools: crt.sh, CertSpotter, Facebook CT Monitor.

Tip 6: Combine DNS with HTTP Analysis

A subdomain resolving to an IP is not enough. Check the HTTP response with the correct Host header. An IP may host multiple applications, and the wrong Host header may return a 404 even though the subdomain "resolves."

Tip 7: Check Historical DNS Regularly

DNS changes over time. A secure configuration today may have been vulnerable yesterday. Historical DNS data from SecurityTrails can reveal past takeovers that were fixed - report these as "Fixed but worth auditing."

Tip 8: Don't Ignore NSEC Walking

DNSSEC NSEC records allow zone enumeration even when zone transfer is disabled. If you find DNSSEC with NSEC (not NSEC3), you can enumerate all subdomains without brute-forcing. This is a reconnaissance goldmine.

Responsible Disclosure for DNS Takeovers

When you successfully claim a subdomain, do NOT serve malicious content, collect user data, or intercept traffic. Serve a simple HTML proof-of-concept with your researcher name and contact. Immediately report and release the resource after acknowledgment.

Resource	URL
crt.sh	https://crt.sh
SecurityTrails	https://securitytrails.com
DNSViz	https://dnsviz.net
ZoneMaster	https://zonemaster.net
MXToolbox	https://mxtoolbox.com
DMARC Analyzer	https://dmarcian.com
Google Admin Toolbox	https://toolbox.googleapps.com
Mail-Tester	https://mail-tester.com
ViewDNS	https://viewdns.info
IPInfo	https://ipinfo.io